

Unity & Arduino Serial Communication – Student Guide

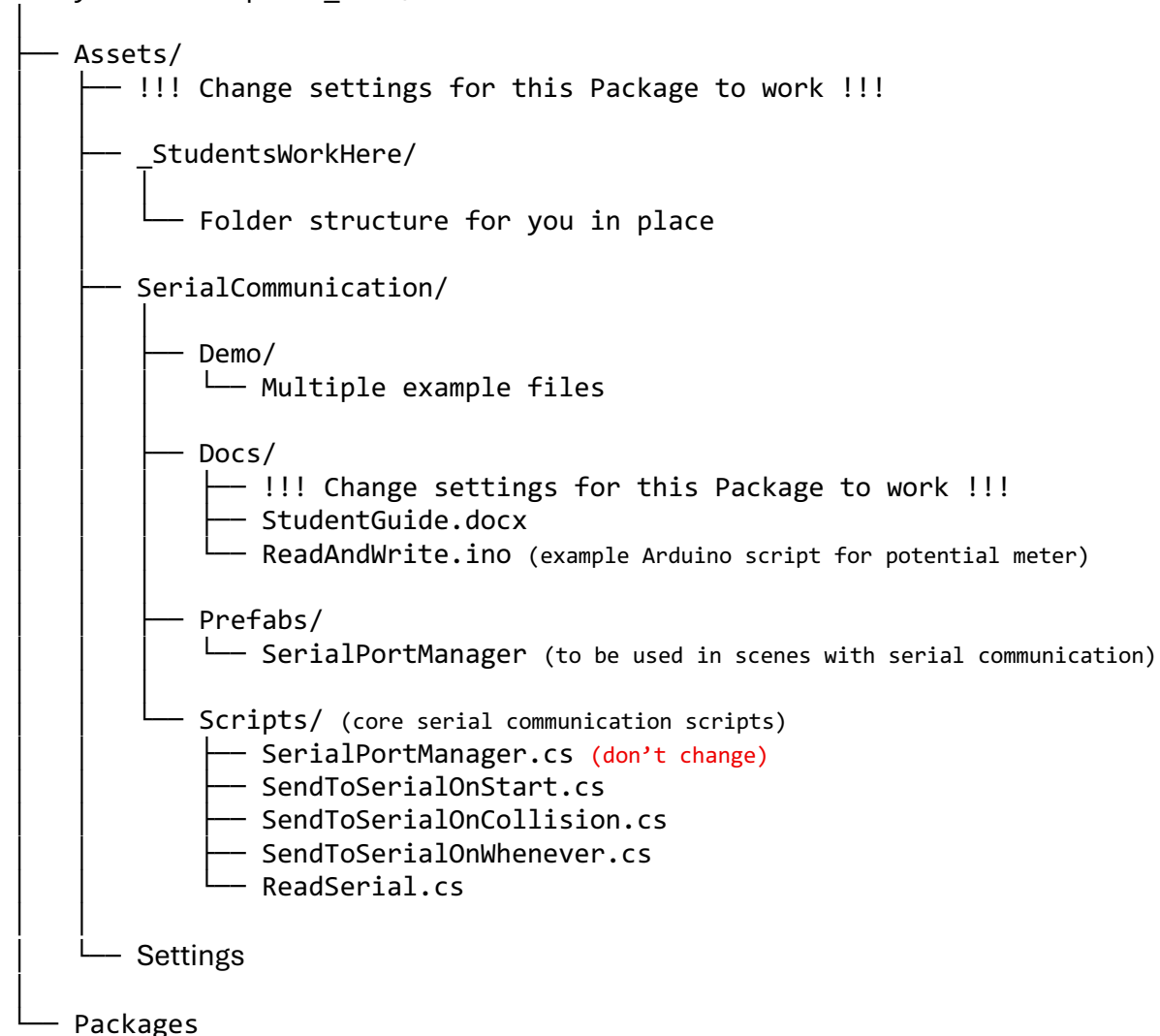
1. Introduction

This guide explains how to use Unity to communicate with an Arduino via a serial port. Students will learn: - How to receive data from Arduino sensors in Unity. - How to send messages from Unity to Arduino actuators or events. - How to use an event-driven system with SerialPortManager.

2. Project Structure

The template project is organized as follows:

UnityArduinoTemplate_2026/



3. Make connection with Arduino

- Change project settings before or immediately after importing package:
Menu <Edit><Project Settings>
<player><other settings><API Compatibility Level*> [.NET Framework](#)
 - Connect your Arduino
 - In the Arduino IDE, find the port name you connected the Arduino to and note your port name
For windows it will be something like: COM1, COM2, etc.
For Mac it will be something like: /dev/cu.usbmodem12401
 - Open the Scene in the Student folder called: Student_Scene
 - On the GameObject SerialPortManager in the scene change in the inspector the port name into the name on which your Arduino is Connected
 - Be sure that your Serial Monitor in the Arduino IDE is closed
 - Test if the connection with the Serial Port is made by pressing Play and watching the console and look at the SerialPortManager to see the values received (if you already made an Arduino setup that sends out data)
-

4. Arduino Example (ArduinoExample.ino)

```
const int led1 = 12; // Pin for first LED
const int led2 = 13; // Pin for second LED
bool running = false; // Flag to check if blinking should happen
int potPin = A0; // Analogue pin to which the potentiometer is connected
int potValue; // Potentiometer value

void setup() {
    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);
    Serial.begin(9600); // Start serial communication
    delay(2000); // Wait until USB is stable
    Serial.println("READY"); // Notify Unity that Arduino is ready
}

void loop() {
    if (Serial.available() > 0) {

        char command = (char)Serial.read();
        if (command == 'A') {
            running = true;
        } else if (command == 'B') {
            running = false;
            digitalWrite(led1, LOW);
        }
    }
}
```

```

        digitalWrite(led2, LOW);
    }
}

if (running) {
    static unsigned long lastToggleTime = 0;
    static bool ledState = false;

    if (millis() - lastToggleTime >= 500) {
        lastToggleTime = millis();
        ledState = !ledState;
        digitalWrite(led1, ledState);
        digitalWrite(led2, !ledState);
    }
}

potValue = analogRead(potPin); // Read the potentiometer value
Serial.println(potValue); // Send the value to the serial port
delay(50); // Slight delay for more stable values
}

```

- Students can replace this with other sensors or actuators.

5. SerialPortManager.cs

- Central manager for all serial communication.
- Responsibilities:
 - Opening the port.
 - Sending safely (SendSafe();).
 - Reading data in a background thread.
 - Event system: OnRawDataReceived and OnPortReady.

Key Methods & Events:

- SendSafe(string message) – safely sends messages with cooldown.
- OnRawDataReceived – subscribe to receive new Arduino data.
- OnPortReady – subscribe to send messages as soon as Arduino is ready.

Students should **not modify** this script unless adding advanced features.

6. Sending Messages

- **SendToSerialOnStart.cs** – sends a message automatically when the port is ready.
- **SendToSerialOnCollision.cs** – sends a message when an object this script is on collides with an other object with the tag “hit”.

- **SendToSerialOnWhenever.cs** – allows calling `SendToSerial()` when the student decides it should send, so the script should be modified to send `WhenSomethingHappens`.

All scripts use `SendSafe()` to ensure messages are sent only when the port is ready and respecting cooldown.

7. Receiving Data

- **ReadSerial.cs** subscribes to `OnRawDataReceived`.
- Example processing, this one remaps the data from 0-1023 to 0-100:

```
void ProcessRawData(string raw) {  
    if (int.TryParse(raw, out int sensorVal)) {  
        mappedValue = Mathf.Lerp(0f, 100f, sensorVal / 1023f);  
    }  
}
```

- Students can create their own scripts to process data independently.
-

8. Demo Scene

- `SpeedByPot.unity` shows:
 - Example object moving with potentiometer input (`DriveCarWithPotmeter.cs`)
 - Example objects sending messages.
 - Different objects reacting to the potmeter
 - Students can inspect how scripts are attached and interact.
-

9. Workflow for Students

1. Run the demo scene to verify communication.
 2. Use the folder `_StudentsWorkHere/` to create your scripts and put your work here.
 3. Subscribe to `OnRawDataReceived` to use Arduino data.
 4. Call `SendSafe()` or change `SendToSerialOnWhenever.SendToSerial()` to send messages.
-

10. Notes & Tips

- Do **not modify** `SerialPortManager.cs` unless extending functionality.
- Test communication with Arduino connected.
- Use `Debug.Log` to view raw values.
- Use `OnPortReady` to send messages immediately after Arduino signals `READY`.
- You can replace the example potentiometer with any sensor or actuator.